

Base de Conocimiento Técnica — Sistema UR5 + RG2 Pick & Place

Versión: 1.0 — 2026-04-18

Workspace: `/home/laboratorio/TFM/agarre_ros2_ws`

Stack: ROS 2 Jazzy · Gazebo Sim · MoveIt 2 · ros2_control · Python 3.12

Autor del análisis: Claude Sonnet 4.6 (revisión automática del código fuente real)

1. Resumen Ejecutivo del Sistema

1.1 Qué hace el proyecto

Este proyecto implementa un sistema completo de **pick & place** simulado con un robot UR5 equipado con el gripper OnRobot RG2. El sistema selecciona un objeto cilíndrico de una mesa, lo agarra con precisión y lo transporta a una cesta destino. Toda la lógica opera en simulación Gazebo con tiempo de simulación, sin hardware real.

Flujo operativo de alto nivel:

```
Lanzar sistema → Panel Qt → Seleccionar objeto → Run Pick Demo →
HOME → APPROACH → GRASP_DOWN → GRASP_ALIGN_IK → PRE_CLOSE →
CLOSE → ATTACH_GATE → LIFT → TRANSPORT → CESTA → HOME_FINAL
```

1.2 Componentes principales

Componente	Rol
UR5 (6 DOF)	Brazo robótico universal
OnRobot RG2	Pinza paralela de 2 fingers
Gazebo Sim	Simulación física + sensores
MoveIt 2	Planificación de trayectorias
ros2_control	Controladores de joints
Panel Qt (panel_v2.py)	Interfaz de usuario + orquestación
ur5_moveit_bridge	Puente peticiones panel → MoveIt
gripper_attach_backend	Gestión de sujeción virtual (detachable joints)
Cámara overhead	Visión para detección de objetos

1.3 Stack tecnológico

- **ROS 2 Jazzy** (Python rclpy + C++ donde necesario)
- **Gazebo Sim** (gz-sim 8.x) con plugins SDF

- **Movelt 2** (move_group + computeCartesianPath)
- **ros2_control** (joint_trajectory_controller + gripper_controller)
- **Python 3.12** para toda la lógica del panel
- **Qt5/PyQt5** para la interfaz gráfica
- **IK solver propio** (ur5_kinematics.py, DH params UR5)

1.4 Cómo se lanza

```
cd /home/laboratorio/TFM/agarre_ros2_ws
source install/setup.bash

# Lanzar stack completo
ros2 launch ur5_bringup ur5_stack.launch.py

# El panel Qt arranca automáticamente
# Alternativamente, lanzar panel por separado:
ros2 run ur5_qt_panel panel_v2
```

Variables de entorno de configuración se inyectan en el launch file o mediante export antes del launch. Ver Sección 6.

1.5 Flujo operativo general

1. Sistema arranca: Gazebo, RSP, controladores, Movelt, panel
2. Usuario abre panel Qt
3. Cámara overhead detecta objeto en mesa
4. Usuario selecciona objeto en UI
5. Usuario pulsa "Run Pick Demo"
6. El panel ejecuta ciclo secuencial de fases (Sección 5)
7. Al finalizar: objeto en cesta, robot en HOME_FINAL

2. Arquitectura del Proyecto

2.1 Paquetes ROS 2

Paquete	Ruta	Responsabilidad
ur5_bringup	src/ur5_bringup/	Launch files, configuración de controladores
ur5_description	src/ur5_description/	URDF/Xacro del robot UR5 + RG2
ur5_moveit_config	src/ ur5_moveit_config/	Configuración Movelt 2 (SRDF, kinematics, pipelines)
ur5_qt_panel	src/ur5_qt_panel/	Panel Qt, lógica de pick demo, módulos de geometría
tfm_grasping	src/tfm_grasping/	Percepción e inferencia de grasp (visión)
ur5_tools	src/ur5_tools/	

Paquete	Ruta	Responsabilidad
		Herramientas auxiliares (system_state, attach_backend)

2.2 Nodos principales

Nodo	Archivo fuente	Función
<code>robot_state_publisher</code>	(ros2 built-in)	Publica /robot_description + árbol TF
<code>gz_sim</code>	(gz-sim built-in)	Motor de simulación Gazebo
<code>ros_gz_bridge</code>	(ros_gz built-in)	Puente topics Gazebo ↔ ROS 2
<code>joint_trajectory_controller</code>	(ros2_control)	Ejecuta trayectorias del brazo
<code>gripper_controller</code>	(ros2_control)	Controla apertura/cierre pinza
<code>joint_state_broadcaster</code>	(ros2_control)	Publica <code>/joint_states</code>
<code>gz_pose_bridge</code>	<code>ur5_tools/gz_pose_bridge</code>	Poses Gazebo → TF ROS 2
<code>world_tf_publisher</code>	<code>ur5_tools/world_tf_publisher</code>	TF estático world → base_link
<code>system_state_manager</code>	<code>ur5_tools/system_state_manager</code>	Monitor estado global del sistema
<code>gripper_attach_backend</code>	<code>ur5_tools/gripper_attach_backend</code>	Gestión detachable joints Gazebo
<code>planning_scene_sync</code>	<code>ur5_tools/planning_scene_sync</code>	Sincroniza escena MoveIt con Gazebo
<code>ur5_moveit_bridge</code>	<code>ur5_tools/ur5_moveit_bridge</code>	Puente /desired_grasp → MoveIt
<code>panel_v2</code>	<code>ur5_qt_panel/panel_v2.py</code>	Interfaz Qt principal + orquestación

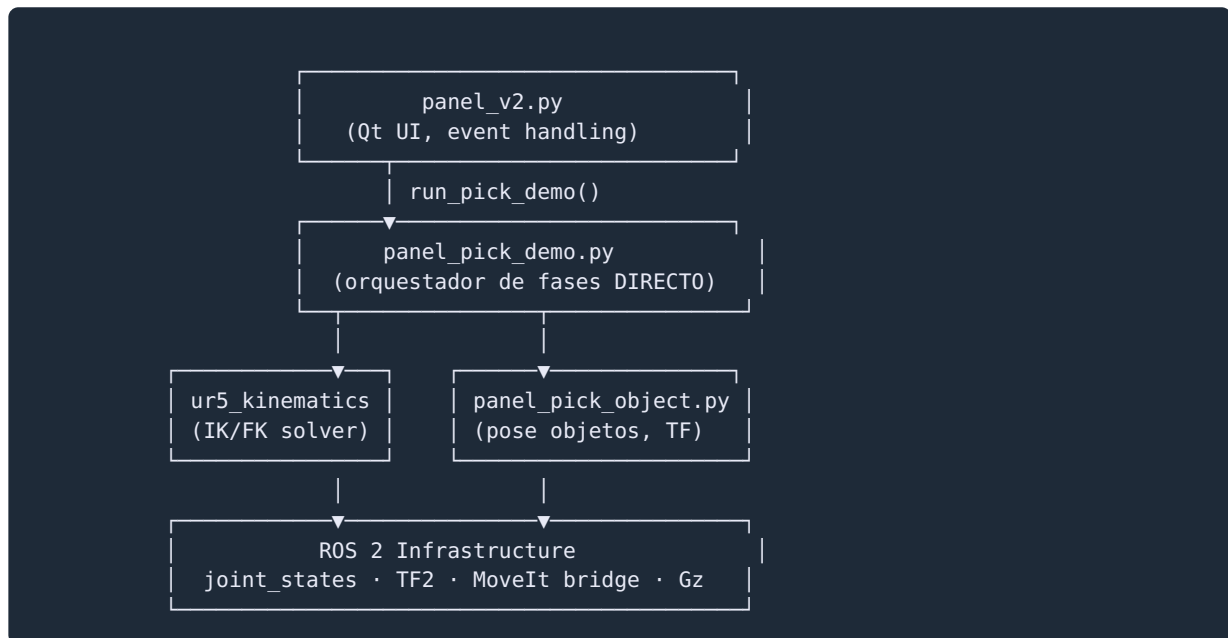
2.3 Launch files relevantes

Archivo	Ruta	Qué hace
<code>ur5_stack.launch.py</code>	<code>src/ur5_bringup/launch/</code>	Launch principal: todo el stack completo
<code>ur5_rsp.launch.py</code>	<code>src/ur5_bringup/launch/</code>	Solo robot_state_publisher
<code>ur5_ros2_control.launch.py</code>	<code>src/ur5_bringup/launch/</code>	Solo controladores ros2_control
<code>ur5_moveit_bringup.launch.py</code>	<code>src/ur5_moveit_config/launch/</code>	Solo MoveIt 2 (move_group)

2.4 Qué archivo controla cada parte

Parte del sistema	Archivo de control
Frames TF del robot	<code>src/ur5_description/urdf/ur5.urdf.xacro</code>
Modelo físico Gazebo	<code>models/ur5_rg2/model.sdf</code>
Mundo de simulación	<code>worlds/ur5_mesa_objetos.sdf</code>
Controladores	<code>src/ur5_bringup/config/ur5_mock_controllers.yaml</code>
Configuración MoveIt	<code>src/ur5_moveit_config/config/</code>
Variables de entorno pick	<code>src/ur5_bringup/launch/ur5_stack.launch.py</code>
Lógica pick demo (fases)	<code>src/ur5_qt_panel/ur5_qt_panel/panel_pick_demo.py</code>
Orquestación UI	<code>src/ur5_qt_panel/ur5_qt_panel/panel_v2.py</code>
Detección de objetos	<code>src/ur5_qt_panel/ur5_qt_panel/panel_pick_object.py</code>
Parámetros globales	<code>src/ur5_qt_panel/config/panel_settings.yaml</code>
Gate ATTACH	<code>src/ur5_qt_panel/ur5_qt_panel/attach_gate_evaluator.py</code>
IK/FK custom	<code>src/ur5_qt_panel/ur5_qt_panel/ur5_kinematics.py</code>
Geometría de agarre	<code>src/ur5_qt_panel/ur5_qt_panel/panel_pick_geometry.py</code>
Estado lógico objetos	<code>src/ur5_qt_panel/ur5_qt_panel/panel_objects.py</code>

2.5 Relación entre módulos



Flujo de un comando de movimiento:

```

panel_pick_demo.py
→ _send_ik_motion(target_pose_base link)
→ ur5_kinematics.ik_ur5(target) [IK directo NEGATE_XY]
→ publish /joint_trajectory_controller/joint_trajectory
→ joint_trajectory_controller ejecuta en Gazebo
→ joint_state_broadcaster publica /joint_states
→ panel lee /joint_states para verificar convergencia

```

Flujo de un comando MoveIt (GRASP_DOWN cartesiano):

```

panel_pick_demo.py
→ _request_moveit_cartesian(waypoints)
→ publish /desired_grasp/request (PoseStamped[])
→ ur5_moveit_bridge recibe request
→ move_group.computeCartesianPath()
→ move_group.execute()
→ joint states convergen
→ ur5_moveit_bridge publica /desired_grasp/result
→ panel_pick_demo.py recibe resultado

```

3. Frames y Offsets

3.1 Tabla completa de frames

Frame	Tipo	Padre	Offset (xyz)	Descripción
<code>world</code>	Gazebo origin	—	—	Origen absoluto de la simulación
<code>base_link</code>	URDF fixed	<code>world</code>	(0, 0, 0.850)	Base del UR5; 850 mm sobre el suelo
<code>base_link_inertia</code>	URDF fixed	<code>base_link</code>	(0, 0, 0) Rz(π)	Raíz cinemática DH — rotada 180° en Z respecto a <code>base_link</code>
<code>shoulder_link</code>	joint	<code>base_link_inertia</code>	per DH	Hombro UR5
<code>upper_arm_link</code>	joint	<code>shoulder_link</code>	per DH	Brazo superior
<code>forearm_link</code>	joint	<code>upper_arm_link</code>	per DH	Antebrazo
<code>wrist_1_link</code>	joint	<code>forearm_link</code>	per DH	Muñeca 1
<code>wrist_2_link</code>	joint	<code>wrist_1_link</code>	per DH	Muñeca 2
<code>wrist_3_link</code>	joint	<code>wrist_2_link</code>	per DH	Muñeca 3
<code>tool0</code>	URDF fixed	<code>wrist_3_link</code>	(0, 0, 0)	TCP del brazo UR5 (sin gripper)
<code>rg2_base_link</code>	URDF fixed	<code>tool0</code>	(0, 0, 0)	Base del gripper RG2
<code>rg2_tcp</code>	URDF fixed	<code>tool0</code>	(0, 0, 0.175)	TCP virtual del gripper (alias de <code>rg2_pinch_center</code>)

Frame	Tipo	Padre	Offset (xyz)	Descripción
<code>rg2_pinch_center</code>	URDF fixed	<code>tool0</code>	(0, 0, 0.175)	Frame operacional de contacto — punto entre fingers
<code>rg2_hand</code>	URDF fixed	<code>rg2_base_link</code>	(0.105, 0, 0)	Cuerpo principal del gripper
<code>rg2_leftfinger</code>	revoluto	<code>rg2_hand</code>	(0.105, 0.017, 0)	Finger izquierdo (joint1)
<code>rg2_rightfinger</code>	revoluto	<code>rg2_hand</code>	(0.105, -0.017, 0)	Finger derecho (joint2)
<code>camera_wrist_link</code>	URDF fixed	<code>rg2_hand</code>	(var)	Cámara montada en gripper
<code>pick_demo_anchor</code>	static	<code>tool0</code>	(0, 0, 0.175)	Ancla para gripper_attach_backend

3.2 Significado y usos de cada frame clave

`world`

- Origen de Gazebo. Todas las poses de objetos simulados se expresan primero en `world`.
- Usado por: `gz_pose_bridge`, `world_tf_publisher`, `panel_pick_object.py` (conversión a `base_link`).
- NUNCA usar directamente en comandos IK del panel.

`base_link`

- Raíz del árbol TF del robot. Publicado por RSP.
- Frame de referencia para **todos los targets IK** que genera el panel.
- Usado por: `panel_pick_demo.py`, `ur5_kinematics.ik_ur5()`, MoveIt, `ur5_moveit_bridge`.

`base_link_inertia`

- Frame interno del solver DH. Tiene rotación $R_z(\pi)$ respecto a `base_link`.
- **⚠ CRÍTICO:** El solver IK (`ur5_kinematics.py`) opera en este frame. Por eso se aplica NEGATE_XY a los targets antes de pasar al solver. Ver Sección 8.4.
- No se referencia explícitamente en el código del panel, pero es el sistema de coordenadas subyacente del IK.

`tool0`

- TCP del brazo UR5 sin gripper. Publicado por RSP vía FK.
- **⚠ Trampa:** La UI históricamente mostraba la pose del TCP como `tool0`. Esto causa confusión porque el contacto físico real ocurre en `rg2_pinch_center`, 175 mm más arriba.
- Usado por: URDF (padre de `rg2_base_link`), MoveIt internamente.

rg2_pinch_center

- **El frame correcto para toda la lógica de grasp.** Punto central entre los dos fingers en posición de cierre.
- Offset respecto a tool0: $z = +0.175$ m (fijo, sin dependencia del estado del gripper).
- Usado por: `panel_pick_demo.py` para todos los targets de APPROACH, GRASP_DOWN, GRASP_ALIGN_IK, CLOSE, ATTACH_GATE.
- Definido en: `src/ur5_description/urdf/ur5.urdf.xacro`.

3.3 Relaciones clave entre frames

```
world
├─ base_link (+0.850 m Z en world)
│   └─ [cadena DH UR5: 6 joints]
│       └─ tool0
│           ├── rg2_tcp (+0.175 m Z) ← alias
│           ├── rg2_pinch_center (+0.175 m Z) ← FRAME OPERACIONAL
│           ├── rg2_base_link (+0.000 m Z)
│           │   └─ rg2_hand (+0.105 m X)
│           │       ├── rg2_leftfinger (revoluto, +Y)
│           │       └─ rg2_rightfinger (revoluto, -Y)
│           └─ pick_demo_anchor (+0.175 m Z) ← usado por attach_backend
```

Transformación objeto world → base_link:

```
# panel_pick_object.py
tf_buffer.lookup_transform("base_link", "world", rclpy.time.Time())
# Aplica: P_base = R_world_to_base @ P_world + t_world_to_base
# Donde t = (0, 0, -0.850) porque base_link está +0.850 sobre world
```

3.4 Errores típicos por mezcla de frames

Error	Síntoma	Causa
Usar <code>tool0</code> en lugar de <code>rg2_pinch_center</code> para target Z	Robot baja 175 mm de más, colisiona con mesa	No añadir offset del gripper al target
Usar coordenadas <code>world</code> como si fueran <code>base_link</code>	Robot va a posición incorrecta ~850 mm desplazada en Z	No aplicar transformación world→base_link
No aplicar NEGATE_XY al pasar target al solver IK	Error de posición especular en XY	base_link_inertia tiene $R_z(\pi)$
Mezclar poses snapshot (TF live) con poses stable cache (panel state)	Divergencia si el objeto se mueve entre actualizaciones	Usar siempre la misma fuente dentro de una fase
Aplicar GRIPPER_TCP_Z_OFFSET sobre rg2_pinch_center	Doble offset: target Z incorrecto +0.05 m adicional	Bug histórico corregido (ver Sección 8)

4. Geometría del Gripper y del Objeto

4.1 Dimensiones útiles del RG2

Parámetro	Valor	Fuente
Apertura máxima (física)	1.18 rad por finger	<code>model.sdf</code> , joint limits
Apertura máxima (mm apertura lineal)	~110 mm estimado	Geometría finger revoluto
Punto TCP operacional	<code>rg2_pinch_center</code> (+0.175 m sobre <code>tool0</code>)	<code>ur5.urdf.xacro</code>
Distancia <code>tool0</code> → contacto real	0.175 m	URDF offset
Fricción fingers (μ)	2.15	<code>model.sdf</code> contact surface
Stiffness contacto (kp)	200,000	<code>model.sdf</code>
Damping contacto (kd)	30.0	<code>model.sdf</code>
Masa gripper completo	~2.0 kg (estimado)	<code>model.sdf</code> inertials

4.2 Punto TCP operativo

El **TCP operativo** del sistema es `rg2_pinch_center`, que se encuentra exactamente **0.175 m en el eje Z positivo** de `tool0`.

Esto significa:

- Cuando el panel calcula un target en `base_link` para que el TCP llegue a una posición (x, y, z), el IK debe posicionar `tool0` en (x, y, z - 0.175).
- El código del panel **ya tiene en cuenta este offset** al calcular los targets. El punto de referencia en todos los cálculos es `rg2_pinch_center`, no `tool0`.

4.3 TCP virtual vs contacto físico real

Aspecto	TCP Virtual (<code>rg2_pinch_center</code>)	Contacto Físico Real
Posición	Exactamente $z = +0.175$ sobre <code>tool0</code>	Depende de apertura del gripper y geometría del finger
Cálculo	Offset fijo URDF (no varía con estado gripper)	Varía con la apertura
Uso en código	Target de todos los movimientos	Solo relevante para análisis de fuerza
Offset residual	0 mm (por definición del URDF)	~0-5 mm según apertura

Nota práctica: En el sistema actual, la alineación final de Z se ajusta mediante el bias loop de `GRASP_ALIGN_IK` para compensar el residual entre el modelo DH y el SDF (ver Sección 8.1).

4.4 Objeto de pick (cilindro)

Parámetro	Valor	Fuente
Forma	Cilindro (circle en OBJECT_SHAPES)	<code>panel_pick_object.py</code>
Pose spawn (world)	(-0.42, 0.00, 0.875)	<code>worlds/ur5_mesa_objetos.sdf</code>
Altura estimada	~50 mm	Inferido de GRASP_CONTACT_Z_OFFSET_M=0.13 m
Radio estimado	~30 mm	Inferido de tolerancias XY
Nombre en Gazebo	<code>pick_demo</code>	<code>models/</code> , <code>ur5_stack.launch.py</code>

Cálculo de target Z para grasp:

```
Z_target_rg2_pinch_center = Z_objeto_base + GRASP_CONTACT_Z_OFFSET_M
                           = Z_objeto_base + 0.13
```

Donde:

$Z_{\text{objeto_base}} = \text{pose_objeto_en_base_link.z}$

GRASP_CONTACT_Z_OFFSET_M = 0.13 m (lleva pinza al ecuador del cilindro)

4.5 Mesa y restricciones del workspace

Parámetro	Valor
Tamaño mesa	768 × 800 mm
Centro mesa (world XY)	(-0.17, 0.00)
Altura mesa (world Z)	~0.850 m (igual que base_link Z)
Z efectiva del objeto en base_link	~0.025 m (objeto está ~25 mm sobre la mesa en frame base)
Margen objeto desde borde	90 mm mínimo
Cesta destino (world)	(-1.30, 0.00, 0.82)
Reach del UR5	0.85 m radio

Nota sobre alturas: La mesa está a la misma Z en `world` que `base_link`. Por lo tanto, un objeto sobre la mesa tiene Z positiva pequeña en `base_link` (la superficie de la mesa está en $z \approx 0$ en `base_link`, y el objeto tiene su centro a ~25 mm sobre ella).

5. Flujo Completo del Pick

5.1 Diagrama de fases



5.2 Fase HOME

Campo	Detalle
Objetivo	Llevar el robot a posición articular de reposo segura
Target	<code>JOINT_TABLE_POSE_RAD</code> (constante en <code>panel_robot_presets.py</code>)
Frame	Joint space (no cartesiano)
Método	Preset articular vía <code>joint_trajectory_controller</code>
Criterio éxito	Joints convergen dentro de tolerancia articular
Criterio fallo	Timeout de movimiento (150 s por defecto)
Criterio skip	Robot ya está en HOME (check pre-fase)
Logs	<code>[PICK][HOME] Moving to home preset</code>
Env vars	<code>PANEL_MOVEIT_BRIDGE_EXECUTE_TIMEOUT_SEC=150.0</code>

5.3 Fase APPROACH_COARSE

Campo	Detalle
Objetivo	Posicionar TCP (<code>rg2_pinch_center</code>) sobre el objeto con margen de seguridad
Target	<code>(x_obj, y_obj, z_obj_base + APPROACH_COARSE_EXTRA_Z_M)</code> en <code>base_link</code>
Frame	<code>base_link</code> (target IK), <code>rg2_pinch_center</code> (TCP)
Método	IK directo (<code>ur5_kinematics.ik_ur5</code>)
Criterio éxito	Gate: $XY < 12 \text{ mm}$, $Z < 12 \text{ mm}$ (<code>APPROACH_COARSE_GATE_XY/Z_TOL_M</code>)
Criterio fallo	IK no converge o error > umbral tras settle
Criterio skip	Ninguno (siempre se ejecuta)
Logs	<code>[PICK][DIRECT][APPROACH_COARSE] target=... gate_xy=... gate_z=...</code>
Env vars	<code>PANEL_PICK_DEMO_APPROACH_COARSE_EXTRA_Z_M=0.035</code> , <code>PANEL_PICK_DEMO_APPROACH_COARSE_GATE_XY_TOL_M=0.012</code> , <code>PANEL_PICK_DEMO_APPROACH_COARSE_GATE_Z_TOL_M=0.012</code>

Lógica de target:

```
target_z = object_z_in_base_link + APPROACH_COARSE_EXTRA_Z_M # +35 mm sobre objeto
target = (object_x, object_y, target_z) # en base_link
# Convertir a frame IK: negate XY
ik_result = ik_ur5((-target_x, -target_y, target_z - 0.175), seed=current_joints)
```

5.4 Fase GRASP_DOWN

Campo	Detalle
Objetivo	Descender el TCP hasta la altura de agarre (z_{contact}) de forma controlada
Target	$(x_{\text{obj}}, y_{\text{obj}}, z_{\text{obj_base}} + \text{GRASP_CONTACT_Z_OFFSET_M})$ en base_link
Frame	base_link
Método	Movelt <code>computeCartesianPath</code> (si <code>GRASP_DOWN_USE_MOVEIT_CARTESIAN=1</code>) o IK segmentado (5 mm/step)
Criterio éxito	$XY < 8 \text{ mm}$, $Z < 8 \text{ mm}$, $\text{dist3D} < 12 \text{ mm}$ (STRICT tolerances)
Criterio fallo	Error > umbral tras 4 intentos, IK no converge en segmento
Criterio skip	No hay skip
Logs	<code>[PICK][DIRECT][GRASP_DOWN] segment=N/7 target_z=... xy_err=... z_err=...</code>
Env vars	Ver tabla completa en Sección 6

Lógica IK segmentada:

```
# Divide el descenso en segmentos de GRASP_DOWN_SEGMENT_Z_STEP_M (5 mm)
# Aproximadamente 7 segmentos para bajar ~35 mm
for seg_z in np.arange(approach_z, contact_z, -0.005):
    ik_result = ik_ur5(target_at_seg_z, seed=prev_joints, weight=0.65)
    if validate(ik_result):
        execute(ik_result)
        prev_joints = ik_result
    else:
        retry (max 4 attempts per segment)
```

Por qué segmentado: Evitar cambios de "rama" de solución IK. Con steps > 17 mm, el solver puede saltar a otra configuración articular causando errores de hasta 48 mm en Y. 5 mm/step mantiene el solver en la misma rama.

5.5 Fase GRASP_ALIGN_IK

Campo	Detalle
Objetivo	Compensar el residual Z sistemático entre modelo DH y SDF (~13 mm)
Target	$(x_{\text{obj}}, y_{\text{obj}}, z_{\text{contact}})$ con bias Z ajustado iterativamente
Frame	base_link
Método	Loop IK con bias adaptativo en Z
Criterio éxito	$XY < 10 \text{ mm}$ (<code>ALIGN_EXIT_XY_TOL_M</code>) y $Z < 10 \text{ mm}$ (<code>ALIGN_EXIT_Z_TOL_M</code>)
Criterio fallo	Más de 3 intentos sin convergencia

Campo	Detalle
Criterio skip	Si XY y Z ya cumplen desde GRASP_DOWN
Logs	[PICK][DIRECT][GRASP_ALIGN_IK] attempt=N z_err=... bias=... align_ok=True/False
Env vars	PANEL_PICK_DEMO_ALIGN_IK_ERR_TOL=0.08 , PANEL_PICK_DEMO_ALIGN_EXIT_XY_TOL_M=0.010 , PANEL_PICK_DEMO_ALIGN_EXIT_Z_TOL_M=0.010 , PANEL_PICK_DEMO_ALIGN_Z_RESIDUAL_TOL_M=0.008

Lógica bias loop:

```

target_z = z_contact # objetivo nominal
for attempt in range(3):
    execute_ik(target_x, target_y, target_z)
    real_z = fk_rg2_pinch_center_z() # FK real del TCP
    z_error = real_z - z_contact # error residual
    if abs(z_error) > ALIGN_Z_RESIDUAL_TOL (8mm):
        target_z -= z_error # bias: corrige el target
    if xy_error < 10mm and abs(z_error) < 10mm:
        align_ok = True
        break

```

5.6 Fase PRE_CLOSE

Campo	Detalle
Objetivo	Validación final de alineación antes de ejecutar cierre de pinza
Target	Pose actual del TCP vs objeto
Frame	base_link
Método	Lectura FK + comparación con pose objetivo
Criterio éxito	XY < 10 mm, Z_err < 10 mm (PRE_CLOSE_XY/Z_ERR_TOL_M)
Criterio fallo	Error fuera de umbral tras reintentos
Criterio skip	PANEL_PICK_DEMO_SKIP_ALIGN_IF_REACHABLE=1 (si configurado)
Logs	[PICK][DIRECT][PRE_CLOSE] xy_err=... z_err=...
Env vars	PANEL_PICK_DEMO_PRE_CLOSE_XY_TOL_M=0.010 , PANEL_PICK_DEMO_PRE_CLOSE_Z_ERR_TOL_M=0.010 , PANEL_PICK_DEMO_PRE_CLOSE_REALIGN_RETRIES=2

5.7 Fase CLOSE

Campo	Detalle
Objetivo	Cerrar el gripper RG2 hasta la posición de agarre
Target	<code>rg2_finger_joint1 = rg2_finger_joint2 = 1.18 rad</code>
Frame	Joint space (gripper)
Método	Comando <code>gripper_controller</code> + espera confirmación delta joints
Criterio éxito	Suma delta joints > <code>CLOSE_MIN_DELTA_SUM</code> (0.01 m) en < timeout
Criterio fallo	Timeout (<code>CLOSE_CONFIRM_TIMEOUT_SEC=3.0 s</code>) sin delta suficiente
Criterio skip	Ninguno
Logs	<code>[PICK][DIRECT][CLOSE] delta_sum=... confirmed=True/False</code>
Env vars	<code>PANEL_PICK_DEMO_CLOSE_CONFIRM_TIMEOUT_SEC=3.0</code> , <code>PANEL_PICK_DEMO_CLOSE_MIN_DELTA_SUM=0.01</code> , <code>PANEL_PICK_DEMO_GRIPPER_TARGET_TOL_M=0.12</code> , <code>PANEL_PICK_DEMO_CLOSE_XY_TOL_M=0.008</code> , <code>PANEL_PICK_DEMO_CLOSE_Z_ERR_TOL_M=0.008</code>

Confirmación de cierre:

```
# Leer /joint_states continuamente
# Calcular: delta = |current_j1 - initial_j1| + |current_j2 - initial_j2|
# Confirmar si delta > CLOSE_MIN_DELTA_SUM (0.01 m = 10 mm)
# Timeout si no ocurre en CLOSE_CONFIRM_TIMEOUT_SEC (3 s)
```

5.8 Fase ATTACH_GATE

Campo	Detalle
Objetivo	Verificar que el objeto está bien sujeto antes de levantar
Target	Estabilidad TCP-objeto en ventana temporal
Frame	<code>base_link</code> (distancias TCP↔objeto)
Método	<code>AttachGateEvaluator</code> — ventana temporal de 350 ms con 5+ muestras
Criterio éxito	dist_TCP_obj < 40 mm, drift < 12 mm, gripper cerrado, backend OK
Criterio fallo	Cualquier condición no cumplida en ventana temporal
Criterio skip	Ninguno (gate crítico de seguridad)
Logs	<code>[ATTACH_GATE][CHECK] dist=... drift=... closed=T/F</code> / <code>[ATTACH_GATE][PASS]</code> / <code>[ATTACH_GATE][FAIL]</code>

Campo	Detalle
Env vars	<code>PANEL_PICK_DEMO_ATTACH_XY_TOL_M=0.008</code> , <code>PANEL_PICK_DEMO_ATTACH_Z_TOL_M=0.010</code> , <code>PANEL_PICK_DEMO_ATTACH_FOLLOW_MAX_TCP_DIST_M=0.040</code> , <code>PANEL_PICK_DEMO_ATTACH_MAX_REL_DRIFT_M=0.012</code> , <code>PANEL_PICK_DEMO_ATTACH_STABLE_WINDOW_SEC=0.35</code> , <code>PANEL_PICK_DEMO_ATTACH_MIN_STABLE_SAMPLES=5</code>

Condiciones del gate (todas deben cumplirse):

1. `dist(rg2_pinch_center, objeto) < 40 mm`
2. `drift(objeto, TCP) < 12 mm` en ventana 350 ms
3. `suma_joints_gripper < 20 mm` (gripper cerrado)
4. Backend Gazebo confirma detachable joint creado

5.9 Fase LIFT

Campo	Detalle
Objetivo	Elevar el objeto suficientemente para despejarlo de la mesa
Target	TCP actual + delta_Z (altura de seguridad ~150 mm sobre mesa)
Frame	<code>base_link</code>
Método	IK directo o Movelt cartesiano (subida vertical)
Criterio éxito	FK Z del TCP > umbral de seguridad
Criterio fallo	<code>min_lift_delta</code> no alcanzado (objeto se cae), timeout
Criterio skip	Ninguno
Logs	<code>[PICK][DIRECT][LIFT] lift_delta=... min_lift_delta=0.060</code>
Env vars	<code>min_lift_delta</code> (actualmente 0.060 m, tuneado en 2026-04-07 desde 0.025)

Nota: `min_lift_delta` fue aumentado de 0.025 a 0.060 para evitar falso positivo de grasp donde el sistema detectaba éxito antes de que el objeto se levantara realmente.

5.10 Fase CARRY / TRANSPORT

Campo	Detalle
Objetivo	Transportar objeto desde posición de pick hasta zona de entrega
Target	Pose sobre cesta: <code>(-1.30, 0.00, 0.82 + margen)</code> en world → base_link
Frame	<code>base_link</code>
Método	IK directo o Movelt con waypoints
Criterio éxito	TCP en zona de entrega dentro de tolerancia
Criterio fallo	IK no converge, colisión, objeto se suelta en transporte
Logs	<code>[PICK][DIRECT][TRANSPORT] target=...</code>

5.11 Fase CESTA_RELEASE

Campo	Detalle
Objetivo	Soltar el objeto en la cesta
Target	Gripper abierto (0.0 rad ambos joints)
Frame	Joint space (gripper)
Método	Comando <code>gripper_controller</code> apertura + <code>detach</code> topic Gazebo
Criterio éxito	Gripper abierto + detachable joint eliminado
Criterio fallo	Timeout
Logs	<code>[PICK][DIRECT][CESTA_RELEASE] detach=True</code>

5.12 Fase HOME_FINAL

Campo	Detalle
Objetivo	Retornar a posición de reposo tras completar el ciclo
Target	<code>JOINT_TABLE_POSE_RAD</code> (igual que HOME inicial)
Método	Preset articular
Criterio éxito	Joints en HOME dentro de tolerancia

6. Variables de Entorno y Parámetros Críticos

Las variables se inyectan en `ur5_stack.launch.py` mediante `os.environ.get(VAR, DEFAULT)`. Se pueden sobrescribir con `export VAR=valor` antes de ejecutar el launch.

6.1 Tabla completa de variables de entorno

Geometría y altura de agarre

Variable	Default	Archivo	Fase	Unidad
<code>GRASP_CONTACT_Z_OFFSET_M</code>	0.13	<code>ur5_stack.launch.py</code>	GRASP_DOWN	metros
<code>PANEL_PICK_DEMO_APPROACH_COARSE_EXTRA_Z_M</code>	0.035	<code>ur5_stack.launch.py</code>	APPROACH_COARSE	metros

Variable	Default	Archivo	Fase	Unidad

GRASP_DOWN

Variable	Default	Archivo	Fase	Unidad
PANEL_PICK_DEMO_GRASP_DOWN_SEGMENT_Z_STEP_M	0.005	ur5_stack.launch.py	GRASP_DOWN	metro
PANEL_PICK_DEMO_GRASP_DOWN_USE_MOVEIT_CARTESIAN	1	ur5_stack.launch.py	GRASP_DOWN	bool
PANEL_PICK_DEMO_GRASP_DOWN_IK_SEED_WEIGHT	0.65	ur5_stack.launch.py	GRASP_DOWN	[0,1]
PANEL_PICK_DEMO_GRASP_DOWN_KEEP_XY_TOL_M	0.003	ur5_stack.launch.py	GRASP_DOWN	metro
PANEL_PICK_DEMO_GRASP_DOWN_STRICT_XY_TOL_M	0.008	ur5_stack.launch.py	GRASP_DOWN	metro
PANEL_PICK_DEMO_GRASP_DOWN_STRICT_Z_TOL_M	0.008	ur5_stack.launch.py	GRASP_DOWN	metro
PANEL_PICK_DEMO_GRASP_DOWN_STRICT_DIST_TOL_M	0.012	ur5_stack.launch.py	GRASP_DOWN	metro
PANEL_PICK_DEMO_GRASP_DOWN_MAX_ATTEMPTS	4	ur5_stack.launch.py	GRASP_DOWN	int

Variable	Default	Archivo	Fase	Unidad

GRASP_ALIGN_IK

Variable	Default	Archivo	Fase	Unidades
PANEL_PICK_DEMO_ALIGN_IK_ERR_TOL	0.08	ur5_stack.launch.py	GRASP_ALIGN	metros
PANEL_PICK_DEMO_ALIGN_EXIT_XY_TOL_M	0.010	ur5_stack.launch.py	GRASP_ALIGN	metros
PANEL_PICK_DEMO_ALIGN_EXIT_Z_TOL_M	0.010	ur5_stack.launch.py	GRASP_ALIGN	metros
PANEL_PICK_DEMO_ALIGN_Z_RESIDUAL_TOL_M	0.008	ur5_stack.launch.py	GRASP_ALIGN	metros
PANEL_PICK_DEMO_PRE_CLOSE_REALIGN_RETRIES	2	ur5_stack.launch.py	PRE_CLOSE	int

CLOSE

Variable	Default	Archivo	Fase	Unidades	Efecto
PANEL_PICK_DEMO_CLOSE_CONFIRM_TIMEOUT_SEC	3.0	ur5_stack.launch.py	CLOSE	segundos	Timeout para confirmar el cierre del gripper
PANEL_PICK_DEMO_CLOSE_MIN_DELTA_SUM	0.01	ur5_stack.launch.py	CLOSE	metros	Delta mínimo de suma para confirmar el cierre (mm)

Variable	Default	Archivo	Fase	Unidades	Efecto
PANEL_PICK_DEMO_GRIPPER_TARGET_TOL_M	0.12	ur5_stack.launch.py	CLOSE	metros	Tolerancia de objetivo de gripper (mm)
PANEL_PICK_DEMO_CLOSE_XY_TOL_M	0.008	ur5_stack.launch.py	CLOSE	metros	XY tolerancia en CLO
PANEL_PICK_DEMO_CLOSE_Z_ERR_TOL_M	0.008	ur5_stack.launch.py	CLOSE	metros	Z error en CLO

ATTACH_GATE

Variable	Default	Archivo	Fase	Unidades
PANEL_PICK_DEMO_ATTACH_XY_TOL_M	0.008	ur5_stack.launch.py	ATTACH_GATE	metros
PANEL_PICK_DEMO_ATTACH_Z_TOL_M	0.010	ur5_stack.launch.py	ATTACH_GATE	metros
PANEL_PICK_DEMO_ATTACH_FOLLOW_MAX_TCP_DIST_M	0.040	ur5_stack.launch.py	ATTACH_GATE	metros
PANEL_PICK_DEMO_ATTACH_MAX_REL_DRIFT_M	0.012	ur5_stack.launch.py	ATTACH_GATE	metros
PANEL_PICK_DEMO_ATTACH_STABLE_WINDOW_SEC	0.35	ur5_stack.launch.py	ATTACH_GATE	segundos
PANEL_PICK_DEMO_ATTACH_MIN_STABLE_SAMPLES	5	ur5_stack.launch.py	ATTACH_GATE	int
PANEL_PICK_DEMO_ATTACH_MAX_TF_VISUAL_GAP_M	0.020	ur5_stack.launch.py	ATTACH_GATE	metros
ATTACH_BACKEND_MAX_DIST_M	0.06	ur5_stack.launch.py	ATTACH_GATE	metros

Variable	Default	Archivo	Fase	Unidades

PRE_CLOSE

Variable	Default	Fase	Unidades	Efecto
PANEL_PICK_DEMO_PRE_CLOSE_XY_TOL_M	0.010	PRE_CLOSE	metros	Validación XY pre-cierre (10 mm)
PANEL_PICK_DEMO_PRE_CLOSE_Z_ERR_TOL_M	0.010	PRE_CLOSE	metros	Validación Z pre-cierre (10 mm)
PANEL_PICK_DEMO_SKIP_ALIGN_IF_REACHABLE	0	PRE_CLOSE	bool (0/1)	Saltar realineación si pose es alcanzable

APPROACH_COARSE gate

Variable	Default	Fase	Unidades	Efecto
PANEL_PICK_DEMO_APPROACH_COARSE_GATE_XY_TOL_M	0.012	APPROACH	metros	Gate XY approach coarse
PANEL_PICK_DEMO_APPROACH_COARSE_GATE_Z_TOL_M	0.012	APPROACH	metros	Gate Z approach coarse

Pose freshness y fuentes

Variable	Default	Fase	Unidades	Efecto
PANEL_PICK_DEMO_POSE_SOURCE_AGE_TOL_SEC	0.400	Global	segundos	Edad máxima FK/trace válida (400 ms)
PANEL_PICK_DEMO_POSE_SOURCE_TOL_M	0.006	Global	metros	Tolerancia fuente pose (6 mm)
PANEL_PICK_DEMO_PHASE_JUMP_TOL_M	0.010	Global	metros	Tolerancia salto de fase (10 mm)
PANEL_PICK_DEMO_OBJECT_SOURCE_DIVERGENCE_TOL_M	0.150	Global	metros	Umbral divergencia snapshot

Variable	Default	Fase	Unidades	Efecto
				vs stable cache (150 mm)

Settle IK directo

Variable	Default	Fase	Unidades	Efecto
PANEL_PICK_DEMO_DIRECT_IK_RUNTIME_SETTLE_SEC	2.5	IK directo	segundos	Tiempo máximo de settle post-IK
PANEL_PICK_DEMO_DIRECT_IK_RUNTIME_SETTLE_DELTA_M	0.003	IK directo	metros	Delta posición para considerar settled (3 mm)
PANEL_PICK_DEMO_DIRECT_IK_RUNTIME_SETTLE_SAMPLES	3	IK directo	int	Muestras para confirmar settle
PANEL_PICK_DEMO_DIRECT_IK_RUNTIME_SETTLE_POLL_SEC	0.10	IK directo	segundos	Intervalo poll settle (100 ms)

Movelt bridge

Variable	Default	Archivo	Unidades	Efecto
PANEL_MOVEIT_BRIDGE_EXECUTE_TIMEOUT_SEC	150.0	ur5_stack.launch.py	segundos	Timeout ejecución Movelt
PANEL_MOVEIT_BRIDGE_REQUEST_TIMEOUT_SEC	180.0	ur5_stack.launch.py	segundos	Timeout petición total
PANEL_MOVEIT_BRIDGE_VELOCITY_SCALE	0.30	ur5_stack.launch.py	[0,1]	Escala velocidad Movelt (30% conservador)
PANEL_MOVEIT_BRIDGE_ACCEL_SCALE	0.30	ur5_stack.launch.py	[0,1]	Escala aceleración
PANEL_MOVEIT_BRIDGE_JOINT_STATE_TIMEOUT_SEC	6.0	ur5_stack.launch.py	segundos	Timeout joint state
PANEL_MOVEIT_BRIDGE_JOINT_STATE_MAX_AGE_SEC	2.5	ur5_stack.launch.py	segundos	Edad máxima joint state

7. Controladores, Tópicos y Validaciones

7.1 Controladores ros2_control activos

Controlador	Tipo	Joints	Topic comando	Frecuencia
<code>joint_state_broadcaster</code>	broadcaster	todos	—	125 Hz
<code>joint_trajectory_controller</code>	trajectory	shoulder_pan, shoulder_lift, elbow, wrist_1, wrist_2, wrist_3	<code>/</code> <code>joint_trajectory_controller/</code> <code>joint_trajectory</code>	125 Hz
<code>gripper_controller</code>	position	rg2_finger_joint1, rg2_finger_joint2	<code>/gripper_controller/</code> <code>commands</code>	125 Hz

Archivo de configuración: `src/ur5_bringup/config/ur5_mock_controllers.yaml`

```
controller_manager:
  update_rate: 125 # Hz

joint_trajectory_controller:
  joints:
    - shoulder_pan_joint
    - shoulder_lift_joint
    - elbow_joint
    - wrist_1_joint
    - wrist_2_joint
    - wrist_3_joint
  command_interfaces: [position]
  state_interfaces: [position, velocity]

gripper_controller:
  joints:
    - rg2_finger_joint1
    - rg2_finger_joint2
  interface_name: position
```

7.2 Tópicos principales

Topic	Tipo msg	Dirección	Publicador	Suscriptor
<code>/joint_states</code>	<code>sensor_msgs/</code> <code>JointState</code>	PUB	<code>joint_state_broadcaster</code>	panel, Mov
<code>/robot_description</code>	<code>std_msgs/String</code>	PUB	<code>robot_state_publisher</code>	Movelt, rvi
<code>/tf</code>	<code>tf2_msgs/</code> <code>TFMessage</code>	PUB	<code>robot_state_publisher</code> , <code>gz_pose_bridge</code>	todos
<code>/tf_static</code>	<code>tf2_msgs/</code> <code>TFMessage</code>	PUB	<code>world_tf_publisher</code>	todos
<code>/</code> <code>joint_trajectory_controller/</code> <code>joint_trajectory</code>	<code>trajectory_msgs/</code> <code>JointTrajectory</code>	SUB	<code>joint_trajectory_controller</code>	panel (pub
<code>/gripper_controller/</code> <code>commands</code>	<code>std_msgs/</code> <code>Float64MultiArray</code>	SUB	<code>gripper_controller</code>	panel (pub

Topic	Tipo msg	Dirección	Publicador	Suscriptor
<code>/gripper_anchor/pick_demo/attach</code>	<code>std_msgs/Empty</code>	SUB	Gazebo plugin	<code>gripper_</code> (pub)
<code>/gripper_anchor/pick_demo/detach</code>	<code>std_msgs/Empty</code>	SUB	Gazebo plugin	<code>gripper_</code> (pub)
<code>/gripper_anchor/pick_demo/state</code>	Gazebo msg	PUB	Gazebo plugin	<code>attach_g</code>
<code>/desired_grasp/request</code>	<code>geometry_msgs/PoseStamped[]</code>	SUB	<code>ur5_moveit_bridge</code>	panel (pub)
<code>/desired_grasp/result</code>	(moveit result)	PUB	<code>ur5_moveit_bridge</code>	panel (sub)
<code>/clock</code>	<code>roscpp_msgs/Clock</code>	PUB	Gazebo	todos (use)

7.3 Comandos de verificación en runtime

Verificar que los controladores están activos:

```
ros2 control list_controllers
# Esperado: joint_state_broadcaster [active], joint_trajectory_controller [active],
gripper_controller [active]
```

Verificar que el gripper recibe joint_states:

```
ros2 topic echo /joint_states --once
# Buscar rg2_finger_joint1 y rg2_finger_joint2 en la lista
```

Verificar TF world → base_link:

```
ros2 run tf2_ros tf2_echo world base_link
# Esperado: translation (0, 0, 0.850), rotation identity
```

Verificar TCP del gripper:

```
ros2 run tf2_ros tf2_echo base_link rg2_pinch_center
# Muestra la pose actual del TCP operacional
```

Verificar pose del objeto en simulación:

```
ros2 run tf2_ros tf2_echo world pick_demo
# Pose del objeto en Gazebo
```

Mover gripper manualmente:

```
# Abrir gripper
ros2 topic pub /gripper_controller/commands std_msgs/msg/Float64MultiArray \
  "data: [0.0, 0.0]" --once

# Cerrar gripper
ros2 topic pub /gripper_controller/commands std_msgs/msg/Float64MultiArray \
  "data: [1.18, 1.18]" --once
```

Verificar estado attach backend:

```
ros2 topic echo /gripper_anchor/pick_demo/state
```

7.4 Cómo comprobar que el gripper realmente se mueve

El gripper **se mueve físicamente en Gazebo** si:

1. `/joint_states` muestra cambio en `rg2_finger_joint1` y `rg2_finger_joint2`
2. La vista 3D de Gazebo muestra los fingers moviéndose
3. El panel recibe confirmación de `delta_sum > 0.01 m`

Si el gripper no se mueve a pesar del comando:

1. Verificar que `gripper_controller` está `[active]` con `ros2 control list_controllers`
2. Verificar que no hay joint limit violation (joints en 0.0 y comando 0.0 → no mueve)
3. Verificar que el topic `/gripper_controller/commands` tiene suscriptores

7.5 Validar TF y poses en runtime

Árbol TF completo:

```
ros2 run tf2_tools view_frames
# Genera frames.pdf con árbol completo
```

Verificar edad de transforms (staleness):

```
ros2 topic hz /tf
# Si baja frecuencia → TF puede estar stale
```

Verificar que un frame específico está publicándose:

```
ros2 run tf2_ros tf2_monitor world base_link rg2_pinch_center
```


8. Bugs Conocidos y Fixes Aplicados

8.1 Residual DH/SDF de ~13 mm en Z

Campo	Detalle
Síntoma	TCP llega a Z incorrecto (~13 mm alto) tras GRASP_DOWN
Causa raíz	Divergencia entre parámetros DH del solver IK (del paquete <code>ur_description</code>) y la geometría real del SDF de Gazebo
Cómo se detectó	Medición sistemática <code>z_error</code> en GRASP_ALIGN_IK: residual consistente de 13 mm en dirección positiva Z
Fix aplicado	Bias loop en <code>GRASP_ALIGN_IK</code> : ajusta <code>target_z = target_z - z_error</code> iterativamente hasta convergencia
Archivo modificado	<code>src/ur5_qt_panel/ur5_qt_panel/panel_pick_demo.py</code> (fase GRASP_ALIGN_IK)
Parámetros	<code>ALIGN_Z_RESIDUAL_TOL_M=0.008</code> activa bias; <code>ALIGN_EXIT_Z_TOL_M=0.010</code> define convergencia
Impacto	Sistema funcional con residual ~6-7 mm post-bias (dentro del umbral de éxito de 10 mm)

8.2 Falso positivo en detección de grasp (min_lift_delta)

Campo	Detalle
Síntoma	Sistema reportaba GRASP exitoso pero el objeto no estaba sujeto; objeto caía al iniciar LIFT
Causa raíz	<code>min_lift_delta = 0.025 m</code> demasiado bajo: el TCP subía 25 mm pero el objeto (en reposo sobre la mesa) no se levantaba; Gazebo no había creado el detachable joint correctamente
Cómo se detectó	Observación visual en Gazebo: robot subía pero objeto quedaba en mesa
Fix aplicado	<code>min_lift_delta</code> aumentado de 0.025 a 0.060 m. También se añadió función <code>_fresh_gazebo_object_world()</code> para verificar pose fresca del objeto
Archivo modificado	<code>panel_pick_demo.py</code>
Fecha	2026-04-07
Impacto	Eliminado el falso positivo. El sistema ahora verifica que el objeto realmente se eleva 60 mm antes de continuar

8.3 Doble offset Z en gripper (GRIPPER_TCP_Z_OFFSET bug)

Campo	Detalle
Síntoma	TCP bajaba 50 mm de más, llegando casi a la mesa
Causa raíz	

Campo	Detalle
	<code>GRIPPER_TCP_Z_OFFSET = 0.05 m</code> se aplicaba sobre <code>rg2_pinch_center</code> , que ya tiene el offset de 0.175 m respecto a <code>tool0</code> . Double counting del offset.
Cómo se detectó	Inspección de código: el offset se sumaba tanto en el URDF como en el cálculo del target
Fix aplicado	<code>_DIRECTO_GRASP_Z = 0.0</code> (reset del offset manual), eliminando la variable <code>GRIPPER_TCP_Z_OFFSET</code> de los cálculos del panel
Archivo modificado	<code>panel_pick_demo.py</code>
Commit	<code>9b58910</code> (2026-04-08)
Impacto	Target Z correcto. El gripper llega exactamente a <code>Z_objeto + GRASP_CONTACT_Z_OFFSET_M</code>

8.4 NEGATE_XY permanente en solver IK

Campo	Detalle
Síntoma	Si se deshabilitaba la negación, robot iba a posición especular incorrecta
Causa raíz	<code>base_link_inertia</code> tiene rotación $R_z(\pi)$ respecto a <code>base_link</code> . El solver DH de <code>ur5_kinematics.py</code> usa <code>base_link_inertia</code> como raíz cinemática. Para convertir un target de <code>base_link</code> al frame del modelo IK, hay que negar X e Y.
Cómo se detectó	Diagnóstico espacial: poses FK y targets divergían en X e Y de forma especular
Fix aplicado	La negación es permanente y correcta . Se eliminó la variable de entorno <code>PANEL_PICK_DEMO_DIRECT_IK_NEGATE_XY</code> que podía desactivarla accidentalmente.
Archivo modificado	<code>panel_pick_demo.py</code> , <code>ur5_stack.launch.py</code> (nota eliminada en líneas 584-588)
Fecha	2026-04-16
Impacto	IK correcto siempre. No existe forma de desactivar la negación (letra muerta eliminada)

8.5 Divergencia FK/TF-live durante settle (APPROACH_COARSE_NOT_READY)

Campo	Detalle
Síntoma	APPROACH_COARSE falla con "APPROACH_COARSE_NOT_READY": overshoot de 18 mm en error IK y divergencia FK/TF-live de 88-364 mm durante el settle
Causa raíz	H1: <code>ik_err_tol=0.035</code> demasiado estricto para el solver. H2: FK del panel (~220 ms de edad) rechazado por <code>POSE_SOURCE_AGE_TOL_SEC=0.200</code> s — el panel usaba datos obsoletos del FK
Cómo se detectó	Diagnóstico 2026-04-15: medición de <code>z_error</code> y TF age durante ciclo real
Fix aplicado	

Campo	Detalle
	(1) <code>ik_err_tol</code> relajado. (2) <code>PANEL_PICK_DEMO_POSE_SOURCE_AGE_TOL_SEC</code> aumentado de 0.200 a 0.400 s. (3) 7 variables de entorno adicionales tuneadas (settle, tolerancias).
Archivo modificado	<code>ur5_stack.launch.py</code>
Fecha	2026-04-15
Impacto	Primer ciclo completo exitoso a las 19:14-19:15 de 2026-04-15

8.6 Variables de entorno no inyectadas correctamente en launch

Campo	Detalle	
Síntoma	Variables de entorno ignoradas en runtime; sistema usa valores por defecto aunque se hayan exportado	
Causa raíz	En <code>ur5_stack.launch.py</code> , algunas variables se leían con <code>os.environ.get(VAR, DEFAULT)</code> dentro del scope de declaración del nodo, pero la variable había sido exportada después de sourcer el <code>setup.bash</code>	
Fix	Verificar que el export se hace ANTES de ejecutar el launch. Usar <code>`printenv`</code>	<code>grep PANEL_PICK`</code> para confirmar que están en el entorno
Diagnóstico	Añadir <code>print(os.environ.get(VAR))</code> en el inicio del launch para verificar	

8.7 IK cambia de rama en GRASP_DOWN con steps > 17 mm

Campo	Detalle
Síntoma	Error de posición de ~48 mm en eje Y después de un step de GRASP_DOWN
Causa raíz	El solver IK tiene múltiples soluciones ("ramas"). Sin seed fuerte, puede saltar a una rama diferente en cada step, causando un cambio articular brusco que no lleva al punto deseado
Fix	<code>GRASP_DOWN_SEGMENT_Z_STEP_M = 0.005</code> (5 mm). <code>GRASP_DOWN_IK_SEED_WEIGHT = 0.65</code> (65% hacia seed = rama anterior). Con 7 segmentos de 5 mm para bajar 35 mm total, se mantiene la misma rama.
Archivo	<code>ur5_stack.launch.py</code> , <code>panel_pick_demo.py</code>

8.8 CLOSE en estado PEND (gripper no confirma cierre)

Campo	Detalle
Síntoma	La fase CLOSE queda en estado PEND, no avanza a ATTACH_GATE
Posibles causas	(1) Gripper ya estaba cerrado ($\text{delta_sum} \approx 0$). (2) <code>gripper_controller</code> inactivo. (3) Timeout muy corto. (4) Objeto demasiado grande para el gripper.

Campo	Detalle
Diagnóstico	Ver sección 9.1
Fix típico	Verificar estado inicial del gripper (debería estar abierto antes de CLOSE). Verificar <code>ros2 control list_controllers</code> .

9. Guía de Troubleshooting

9.1 CLOSE se queda en PEND

Síntomas: La fase CLOSE no avanza, log muestra `delta_sum=0.000` o `confirmed=False`.

Diagnóstico paso a paso:

```
# 1. Verificar estado actual del gripper
ros2 topic echo /joint_states --once | grep rg2
# Si joints ya están en ~1.18 → gripper ya estaba cerrado antes del comando
# Fix: abrir gripper manualmente antes del ciclo

# 2. Verificar controlador activo
ros2 control list_controllers | grep gripper
# Debe mostrar: gripper_controller [active]

# 3. Verificar que el topic recibe comandos
ros2 topic hz /gripper_controller/commands
# Si no hay mensajes → el panel no está publicando comandos

# 4. Enviar comando manual y observar
ros2 topic pub /gripper_controller/commands std_msgs/msg/Float64MultiArray \
  "data: [1.18, 1.18]" --once
ros2 topic echo /joint_states --once | grep rg2
# Si no cambia → problema en el controlador
```

Causas más comunes y fixes:

Causa	Fix
Gripper ya en posición cerrada	Reiniciar ciclo con gripper en posición abierta (0.0 rad)
<code>gripper_controller</code> inactivo	<code>ros2 control set_controller_state gripper_controller active</code>
Timeout muy corto	Aumentar <code>PANEL_PICK_DEMO_CLOSE_CONFIRM_TIMEOUT_SEC</code>
<code>CLOSE_MIN_DELTA_SUM</code> demasiado alto	Reducir a 0.005 m si objeto es pequeño
Objeto demasiado grande (bloquea gripper)	Verificar tamaño objeto vs apertura máxima

9.2 La pinza no se mueve (gripper inactivo)

```
# Verificar estado completo del sistema ros2_control
ros2 control list_controllers
ros2 control list_hardware_interfaces

# Verificar que Gazebo está corriendo
ros2 topic hz /clock
# Si no hay mensajes → Gazebo no corre

# Verificar bridge Gazebo ↔ ROS 2
ros2 topic list | grep gz
ros2 topic echo /gz/clock --once

# Reiniciar controlador
ros2 control set_controller_state gripper_controller inactive
ros2 control set_controller_state gripper_controller active
```

9.3 La UI muestra poses incoherentes

Síntomas: La posición del TCP en la UI no coincide con la vista de Gazebo.

```
# Verificar TF tree
ros2 run tf2_ros tf2_echo base_link rg2_pinch_center
# Si transformación es errónea → problema en robot_state_publisher o URDF

# Verificar joint_states
ros2 topic echo /joint_states --once
# Si joints no coinciden con posición visual → retraso o desconexión

# Verificar age del TF
ros2 topic hz /tf
# < 10 Hz puede indicar problema

# Reiniciar RSP
# (reiniciar el launch completo si TF está corrupto)
```

Causa frecuente: La pose que muestra la UI proviene del FK calculado con `joint_states` de `/tf`. Si `robot_state_publisher` está lento o hay lag en `use_sim_time`, el FK puede ser stale. Verificar `PANEL_PICK_DEMO_POSE_SOURCE_AGE_TOL_SEC` (400 ms máximo).

9.4 El objeto no se adjunta (ATTACH_GATE falla)

Síntomas: ATTACH_GATE emite `[ATTACH_GATE][FAIL]`, el robot sube pero el objeto queda en la mesa.

```
# Verificar estado del detachable joint
ros2 topic echo /gripper_anchor/pick_demo/state

# Verificar distancia TCP-objeto
ros2 run tf2_ros tf2_echo base_link pick_demo
ros2 run tf2_ros tf2_echo base_link rg2_pinch_center
# Calcular distancia manual

# Verificar que el gripper está cerrado
ros2 topic echo /joint_states --once | grep rg2
# Suma joints debe ser > 0.01 m (cerrado)

# Verificar attach backend
ros2 topic echo /gripper_anchor/pick_demo/attach
# Si no hay mensajes → backend no envía attach
```

Diagnóstico por condición fallida:

Condición	Cómo verificar	Fix
dist_TCP_obj > 40 mm	tf2_echo base_link pick_demo vs rg2_pinch_center	Revisar GRASP_ALIGN_IK, z_contact offset
Gripper no cerrado	joint_states rg2 joints	Ver 9.1
Backend no responde	topic list gripper_anchor	Verificar nodo gripper_attach_backend activo
ATTACH_BACKEND_MAX_DIST_M muy pequeño	—	Aumentar a 0.08 m temporalmente para diagnóstico

9.5 El grasp es débil / objeto cae al inicio del LIFT

Síntomas: ATTACH_GATE pasa, pero al ejecutar LIFT el objeto cae.

Causas probables:

1. **min_lift_delta muy pequeño:** El sistema "confirma" el lift antes de que el objeto realmente se eleve. Verificar que `min_lift_delta = 0.060` (no 0.025).
1. **Detachable joint no creado:** Backend envió attach pero Gazebo no procesó a tiempo. Verificar `/gripper_anchor/pick_demo/state`.
1. **TCP no en contacto real:** El objeto está dentro del umbral de 40 mm pero no entre los fingers. Revisar XY error en GRASP_ALIGN_IK.
1. **Fricción insuficiente en SDF:** En entorno de alta velocidad, la fricción $\mu=2.15$ puede ser insuficiente. Reducir velocidad LIFT.

```
# Verificar que el lift realmente ocurre
ros2 run tf2_ros tf2_echo base_link pick_demo
# Monitorear Z del objeto durante LIFT
# Si Z no sube → objeto no adjunto (bug detachable joint)
# Si Z sube y luego cae → friction issue
```

9.6 El TCP parece correcto pero visualmente no cuadra

Causa más probable: Confusión entre `tool0` y `rg2_pinch_center`.

- `tool0` está 175 mm **por debajo** de `rg2_pinch_center`
- Si la UI muestra `tool0`, la posición real del contacto es 175 mm más alta

```
# Comparar los dos frames
ros2 run tf2_ros tf2_echo base_link tool0
ros2 run tf2_ros tf2_echo base_link rg2_pinch_center
# La diferencia Z debe ser exactamente 0.175 m
```

Si la diferencia no es 0.175 m: Problema en el URDF. Verificar `ur5.urdf.xacro` línea del joint `rg2_pinch_center_joint`.

9.7 Problemas de frames TF

Diagnóstico general:

```
# Ver árbol completo
ros2 run tf2_tools view_frames
evince frames.pdf

# Verificar que world → base_link está publicado
ros2 topic echo /tf_static | grep base_link

# Verificar latencia de transforms
ros2 run tf2_ros tf2_monitor world base_link

# Si falta un frame → verificar que el nodo publicador está activo
ros2 node list | grep -E "rsp|tf_pub|gz_pose"
```

Errores de lookup_transform comunes:

Error	Causa	Fix
<code>ExtrapolationException: target_time</code>	TF demasiado antiguo	Reducir frecuencia de peticiones o aumentar cache time
<code>LookupException: frame does not exist</code>	Nodo RSP caído o URDF sin publicar	Reiniciar robot_state_publisher
<code>ConnectivityException: no connection</code>	Frame no conectado al árbol	Verificar que world_tf_publisher está activo

9.8 Problemas de variables de entorno

Verificar que las variables están correctamente inyectadas:

```
# Antes de lanzar
printenv | grep -E "PANEL_|GRASP_|ATTACH_"

# En el launch, añadir temporalmente:
# import os; print(os.environ.get('GRASP_CONTACT_Z_OFFSET_M', 'NOT_SET'))
```

Problema frecuente: Exportar la variable en una terminal y lanzar en otra. Las variables de entorno no son globales — deben estar en la misma sesión de shell que el launch.

```
# Correcto:
export GRASP_CONTACT_Z_OFFSET_M=0.12
ros2 launch ur5_bringup ur5_stack.launch.py

# Incorrecto (no funciona entre terminales separadas):
# Terminal 1: export GRASP_CONTACT_Z_OFFSET_M=0.12
# Terminal 2: ros2 launch ur5_bringup ur5_stack.launch.py ← no ve la variable
```

9.9 Problemas de controladores

```
# Estado completo
ros2 control list_controllers -v

# Si un controlador está inactive/error:
ros2 control set_controller_state NOMBRE active

# Si el controller_manager no responde:
# → Gazebo probablemente no está corriendo o gz_ros2_control no arrancó
ros2 node list | grep controller_manager

# Verificar que gz_ros2_control_guard funciona:
ros2 topic echo /gz_ros2_control/status # si existe
```

10. Estado Actual del Sistema

10.1 Resuelto y funcional

Ítem	Estado	Fecha fix
Falso positivo grasp (min_lift_delta 0.025→0.060)	✅ Resuelto	2026-04-07
Doble offset Z (GRIPPER_TCP_Z_OFFSET bug)	✅ Resuelto	2026-04-08
Residual DH/SDF 13 mm (bias loop GRASP_ALIGN_IK)	✅ Mitigado (~6-7 mm residual aceptable)	2026-04-15
Divergencia FK/TF-live en APPROACH_COARSE	✅ Resuelto (POSE_SOURCE_AGE_TOL 400 ms)	2026-04-15
IK cambia de rama en GRASP_DOWN	✅ Resuelto (5 mm steps, seed_weight=0.65)	2026-04-15

Ítem	Estado	Fecha fix
NEGATE_XY hardcodeado correctamente	✓ Confirmado y limpiado	2026-04-16
Ciclo completo de pick funcional	✓ Primer ciclo exitoso 2026-04-15 19:14	2026-04-15
DH/SDF mismatch documentado	✓ Diagnosticado (H2/H3 confirmadas)	2026-04-16

10.2 Pendiente / Hipótesis abiertas

Ítem	Estado	Prioridad
Residual Z 6-7 mm (post-bias)	● Aceptado, no crítico. Root cause exacto del mismatch DH/SDF sin investigar a fondo	Baja
Error XY ~4 mm en APPROACH_COARSE	● Aceptado (dentro de umbral 12 mm). No investigado root cause	Baja
Reproducibilidad del ciclo completo (N ciclos consecutivos)	⚠ No validado estadísticamente	Media
Extensión a múltiples objetos	⚠ Panel soporta múltiples objetos pero solo validado con 1 ciclo	Media
Física de contacto finger-objeto en SDF	● Estimada pero no validada con herramienta de análisis de fuerza	Baja
Performance en hardware real (sin Gazebo)	✗ No probado. Proyecto completamente en simulación	Alta (si aplica)
Integración cámara overhead para detección automática	⚠ Infraestructura existe, integración completa no validada	Media

10.3 Próximos pasos recomendados

1. **Validación estadística:** Ejecutar $N > 10$ ciclos consecutivos y medir tasa de éxito, varianza de errores.
1. **Reducir residual DH/SDF:** Investigar calibración exacta de parámetros DH en `ur_description`. Si el SDF usa valores ligeramente diferentes al DH estándar UR5, ajustar los parámetros del solver IK.
1. **Multi-objeto:** Validar el ciclo con 3-5 objetos diferentes en posiciones variadas de la mesa.
1. **Reducir timeouts:** Con el sistema estabilizado, los timeouts conservadores (CLOSE=3s, EXECUTE=150s) pueden reducirse para ciclos más rápidos.
1. **Documentar SRDF MoveIt:** La configuración MoveIt (`src/ur5_moveit_config/config/`) tiene grupos de movimiento y posiciones preset no completamente documentados en este análisis.

Apéndice A: Estructura de Directorios

```
/home/laboratorio/TFM/agarre_ros2_ws/
├── src/
│   ├── ur5_bringup/
│   │   ├── launch/
│   │   │   ├── ur5_stack.launch.py          ← LAUNCH PRINCIPAL
│   │   │   ├── ur5_rsp.launch.py
│   │   │   └── ur5_ros2_control.launch.py
│   │   └── config/
│   │       └── ur5_mock_controllers.yaml
│   ├── ur5_description/
│   │   └── urdf/
│   │       └── ur5.urdf.xacro                ← FRAMES URDF
│   ├── ur5_moveit_config/
│   │   ├── launch/
│   │   │   └── ur5_moveit_bringup.launch.py
│   │   └── config/
│   │       ├── ur5.srdf
│   │       ├── kinematics.yaml
│   │       └── ...
│   ├── ur5_qt_panel/
│   │   ├── ur5_qt_panel/
│   │   │   ├── panel_v2.py                  ← UI PRINCIPAL
│   │   │   ├── panel_pick_demo.py          ← LÓGICA PICK DEMO
│   │   │   ├── panel_pick_object.py        ← GESTIÓN OBJETOS
│   │   │   ├── panel_config.py             ← CONFIGURACIÓN GLOBAL
│   │   │   ├── panel_objects.py            ← ESTADO LÓGICO OBJETOS
│   │   │   ├── panel_pick_geometry.py      ← GEOMETRÍA GRASP
│   │   │   ├── attach_gate_evaluator.py    ← GATE ATTACH
│   │   │   ├── ur5_kinematics.py           ← IK/FK SOLVER
│   │   │   ├── panel_ros_publishers.py     ← PUBLICADORES ROS 2
│   │   │   └── panel_utils.py              ← UTILIDADES
│   │   └── config/
│   │       ├── panel_settings.yaml
│   │       └── panel_test_tuning.yaml
│   ├── tfm_grasping/                        ← VISIÓN / PERCEPCIÓN
│   ├── ur5_tools/
│   │   ├── gripper_attach_backend/
│   │   ├── gz_pose_bridge/
│   │   ├── system_state_manager/
│   │   ├── planning_scene_sync/
│   │   ├── world_tf_publisher/
│   │   └── ur5_moveit_bridge/
│   ├── models/
│   │   └── ur5_rg2/
│   │       └── model.sdf                    ← MODELO GAZEBO ROBOT+GRIPPER
│   ├── worlds/
│   │   ├── ur5_mesa_objetos.sdf            ← MUNDO PRINCIPAL
│   │   └── ur5_debug_empty.sdf
│   ├── historico/                          ← EVIDENCIAS Y AUDITORÍAS
│   └── scripts/                            ← SCRIPTS DE ANÁLISIS
```

Apéndice B: Constantes de Referencia Rápida

Constante	Valor	Significado
<code>tool0 → rg2_pinch_center</code> (Z)	0.175 m	Offset TCP gripper

Constante	Valor	Significado
<code>world → base_link (Z)</code>	0.850 m	Altura robot en simulación
<code>GRASP_CONTACT_Z_OFFSET_M</code>	0.130 m	Offset Z de agarre (ecuador cilindro)
<code>APPROACH_COARSE_EXTRA_Z_M</code>	0.035 m	Margen seguridad sobre objeto
<code>GRASP_DOWN_SEGMENT_Z_STEP_M</code>	0.005 m	Paso descenso segmentado
<code>min_lift_delta</code>	0.060 m	Delta mínimo para confirmar lift
<code>gripper_closed_rad</code>	1.18 rad	Posición joints cerrados
<code>ATTACH_BACKEND_MAX_DIST_M</code>	0.060 m	Distancia max para attach Gazebo
<code>DH/SDF residual Z</code>	~0.013 m	Divergencia conocida modelo
<code>update_rate controllers</code>	125 Hz	Frecuencia ros2_control
<code>CLOSE_CONFIRM_TIMEOUT_SEC</code>	3.0 s	Timeout confirmación cierre
<code>MOVEIT_EXECUTE_TIMEOUT_SEC</code>	150.0 s	Timeout ejecución MoveIt

Documento generado automáticamente a partir del análisis del código fuente real del proyecto.

Fecha: 2026-04-18 | Revisado por: análisis estático del código en `/home/laboratorio/TFM/agarre_ros2_ws`